STOPWATCH · WATCH · TIMER

Basys-3 FPGA 3-in-1 디지털 시계 — 설계 · 시뮬레이션 검증

STOPWATCH WATCH TIMER

2026. 10. 18

팀원 정안성 · 정보윤 · 이종혁

Contents

Basys-3 FPGA 위에 세 가지 시간 기능을 하나의 Top 으로 통합,
Top 테스트벤치로 버튼 → FND 전체 경로를 검증

01 프로젝트 개요
팀 구성 · 기능별 목표 · 핵심 결과

02 시스템 구조
Top 블록 다이어그램 · 신호 흐름

03 기능별 설명 · 시뮬레이션 검증
기능마다 기능 설명 → 시뮬레이션 : 03-1 시계 · 03-2 스톱워치 · 03-3 타이머

04 트러블 슈팅
현상 → 원인 → 해결 → 적용 결과

05 결론 · 고찰
목표 달성 수치 · 남은 과제

06 시연 영상
보드 동작

WATCH

12:00 시작 · SW2 자리 단위 설정 · 12/24 h

STOPWATCH

시작 · 정지 · 초기화 · 업/다운 + LAP

TIMER

+10 초 · +1 분 · +10 분 · 0 도달 알람

01 프로젝트 개요

팀 구성 · 역할 분담

요약 **FND · Top TB / 타이머 · 시계 · 자원 최적화 / 스톱워치 · LAP — 기능 단위로 나눠 설계 · 검증**

정

정안성

FND · Top 검증

1. FND 컨트롤러 설정 자리 점멸 기능 추가
2. Top 모듈 전체 TB 시나리오 구성
3. 트러블 슈팅 2 : 디바운서

FND · TOP TB

정

정보윤

Timer · Watch 설계

1. Timer · Watch 모듈 설계
2. 자원 최적화 — tick 구조 4 종 비교 (LUT · FF)
3. 타이머 검증 · 트러블 슈팅 1

TIMER · WATCH · 자원

이

이종혁

Stopwatch · LAP

1. Stopwatch 기능 동작 확인 (R · L · U)
2. D 버튼 LAP time 추가
3. LAP 저장 · 해제 파형 검증

STOPWATCH · LAP

01 프로젝트 개요

기능별 목표 · 핵심 결과

결과 **목표 기능 전부 구현 — Top TB 검사 447 개 PASS · 보드 동작 확인**

WATCH

SW2 설정 — 커서 + 자리 단위 올림

- SW2 = 1 → 설정 모드, L · R 로 커서 이동
- U · D 로 그 자리만 ± (분 십의 자리 ± 10 분)
- 설정 중인 자리 점멸로 커서 위치 표시

STOPWATCH

기존 스톱워치 + D 버튼 LAP time

- 시작 · 정지 · 초기화 · 업 / 다운 (R · L · U)
- D : 현재 시간 저장 · 표시 고정, 다시 D 로 복귀
- LAP 중에도 내부 측정은 계속

TIMER

새 기능 — 빠른 알람 타이머

- U + 10 초 · R + 1 분 · D + 10 분, 상한 59 : 59
- L 시작 / 진행 중 취소
- 0 도달 → LED 16 개 0.5 s 점멸 알람

447 / 447

Top TB 13 개 · 검사 PASS

1.8 %

LUT 384 · FF 228 (0.5 %)

+3.440 ns

WNS @ 100 MHz · 래치 0

10 s + 20 ns

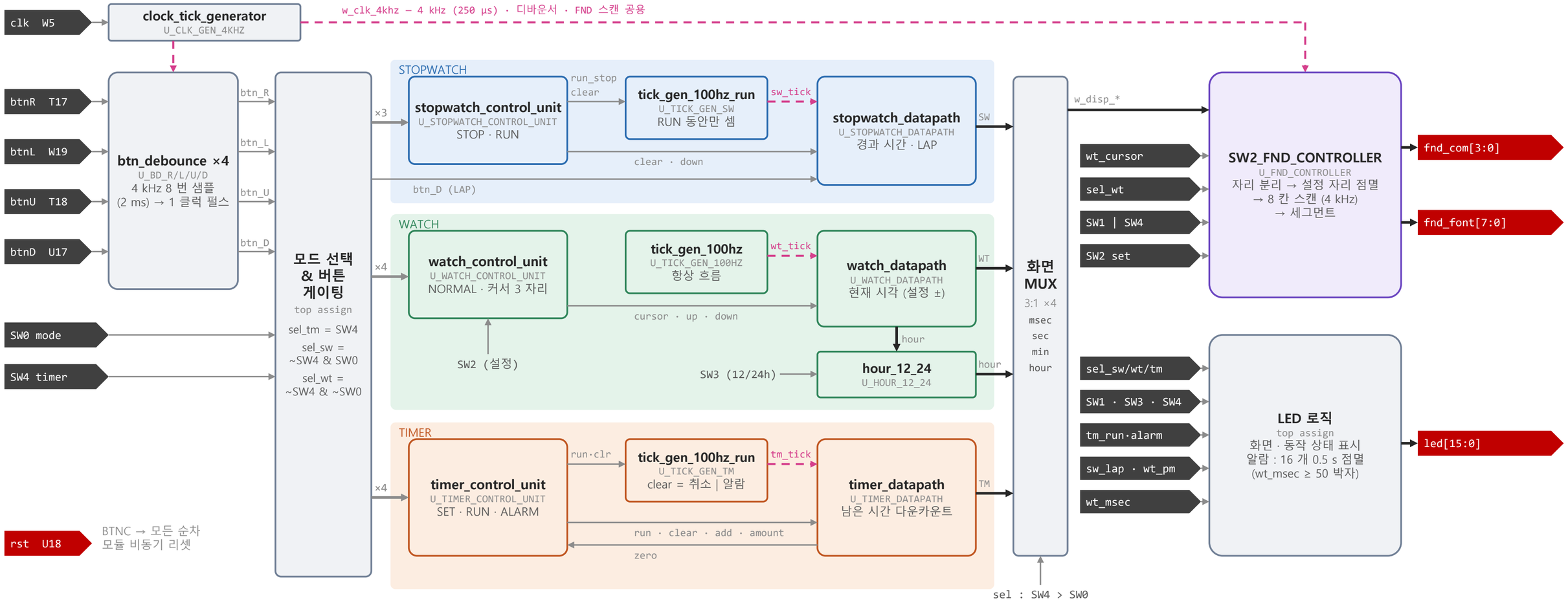
10 초 타이머 실측 (2 클럭 고정 지연)

Note. 수치 출처 — Vivado 2022.2 구현 결과 (xc7a35t) · xsim Top TB [SUMMARY] 자동 판정 (부록 A)

● 02 시스템 구조

Top 블록 다이어그램 — 버튼에서 FND · LED 까지

요약 **기능마다 컨트롤 유닛 → tick → 데이터패스 사슬 · 화면 MUX 로 FND 공유 · 100 Hz tick 은 기능별 분리**



03

기능별 설명 및 시뮬레이션 검증

공통 검증 방법
· Top 모듈을 넣은 TB · 100 MHz 그대로 (주파수 축소 없음)
· 시나리오 표 (입력 → 확인 신호 → 예상 결과) → 파형 · 콘솔 판정

03-1 시계

기능 설명 (조작법) → 시뮬레이션 5 개 (작성 중) · SW2 설정 · 커서 · 자리 단위 증감

03-2 스톱워치

기능 설명 (조작법) → 시뮬레이션 5 개 · R · L · U 기본 동작 + D 버튼 LAP

03-3 타이머

기능 설명 (조작법 · FSM · 구조) → 시뮬레이션 · 더하기 · 59:59 상한 · 카운트다운 · 알람

● 03-1 시계 · 기능 설명

시계 조작법 — 버튼 · 스위치별 동작

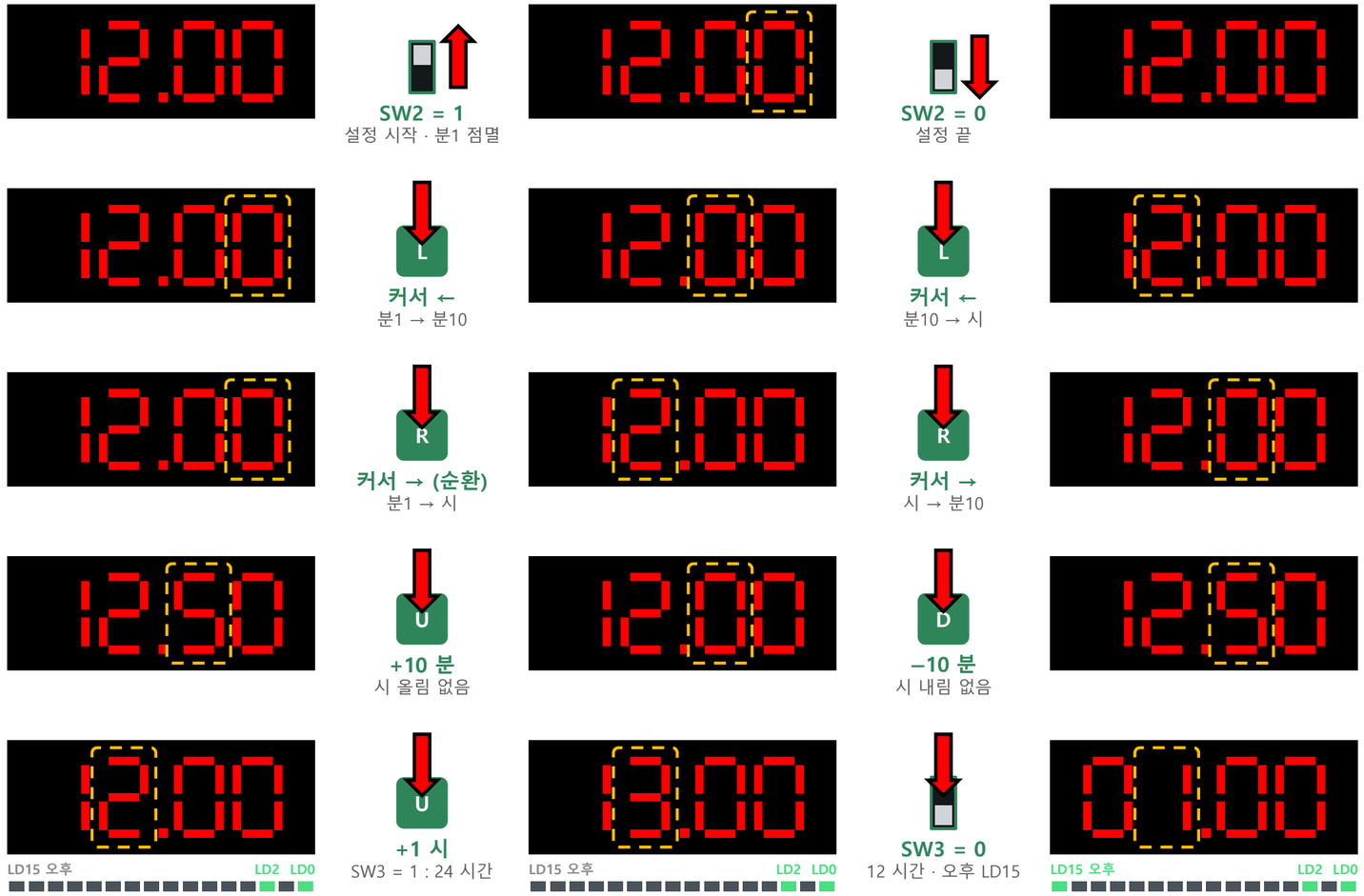
WATCH SW4 = 0 · SW0 = 0 → LD0



#	입력	동작
	SW2	1 : 설정 모드 (분 1 의 자리부터) · 0 : 끝
①②	BTNL / R	설정 자리 이동 시 ↔ 분10 ↔ 분1
③	BTNU	그 자리 + 1 (분10 자리 : + 10 분)
④	BTND	그 자리 – 1 (분10 자리 : – 10 분)
	SW3	0 : 12 시간 (오후 LD15) / 1 : 24 시간
	SW1	1 : 시.분 화면 — 설정 자리 점멸

화면 : SW1 = 1 → 시 . 분 (HH.MM)

■ = 설정 중인 자리 (0.5 s 점멸)



03-1 시계 · 시뮬레이션

시계 시뮬레이션 시나리오

요약 *[작성 중] 시나리오 5 개 — 표 · 파형 추가 예정*

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과
1				
2				
3				
4				
5				

03-1 시계 · 시뮬레이션

시계 시나리오 1:

결과 *[작성 중] 검증 결과 한 줄*

파형 캡처 삽입 영역

① *확인 포인트 (파형 번호와 같은 번호)*

② *확인 포인트 (파형 번호와 같은 번호)*

③ *확인 포인트 (파형 번호와 같은 번호)*

Figure. 시계 시나리오 1 (작성 중)

시계 시나리오 2 :

결과 *[작성 중] 검증 결과 한 줄*

파형 캡처 삽입 영역

① *확인 포인트 (파형 번호와 같은 번호)*

② *확인 포인트 (파형 번호와 같은 번호)*

③ *확인 포인트 (파형 번호와 같은 번호)*

Figure. 시계 시나리오 2 (작성 중)

시계 시나리오 3 :

결과 *[작성 중] 검증 결과 한 줄*

파형 캡처 삽입 영역

① *확인 포인트 (파형 번호와 같은 번호)*

② *확인 포인트 (파형 번호와 같은 번호)*

③ *확인 포인트 (파형 번호와 같은 번호)*

Figure. 시계 시나리오 3 (작성 중)

03-1 시계 · 시뮬레이션

시계 시나리오 4 :

결과 *[작성 중] 검증 결과 한 줄*

파형 캡처 삽입 영역

① *확인 포인트 (파형 번호와 같은 번호)*

② *확인 포인트 (파형 번호와 같은 번호)*

③ *확인 포인트 (파형 번호와 같은 번호)*

Figure. 시계 시나리오 4 (작성 중)

시계 시나리오 5 :

결과 *[작성 중] 검증 결과 한 줄*

파형 캡처 삽입 영역

① *확인 포인트 (파형 번호와 같은 번호)*

② *확인 포인트 (파형 번호와 같은 번호)*

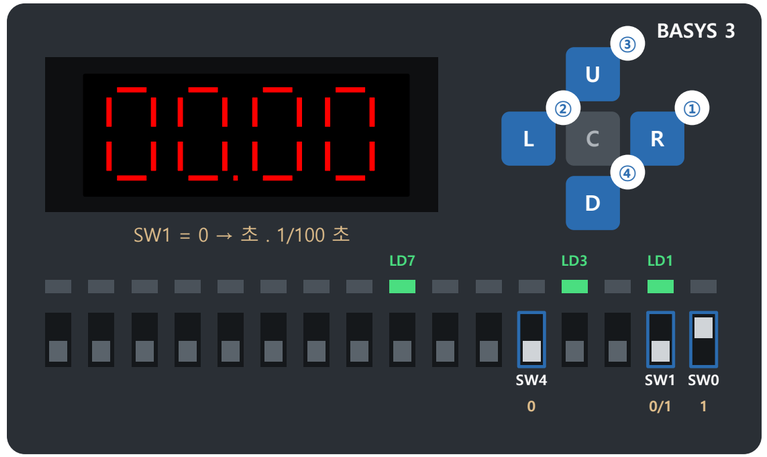
③ *확인 포인트 (파형 번호와 같은 번호)*

Figure. 시계 시나리오 5 (작성 중)

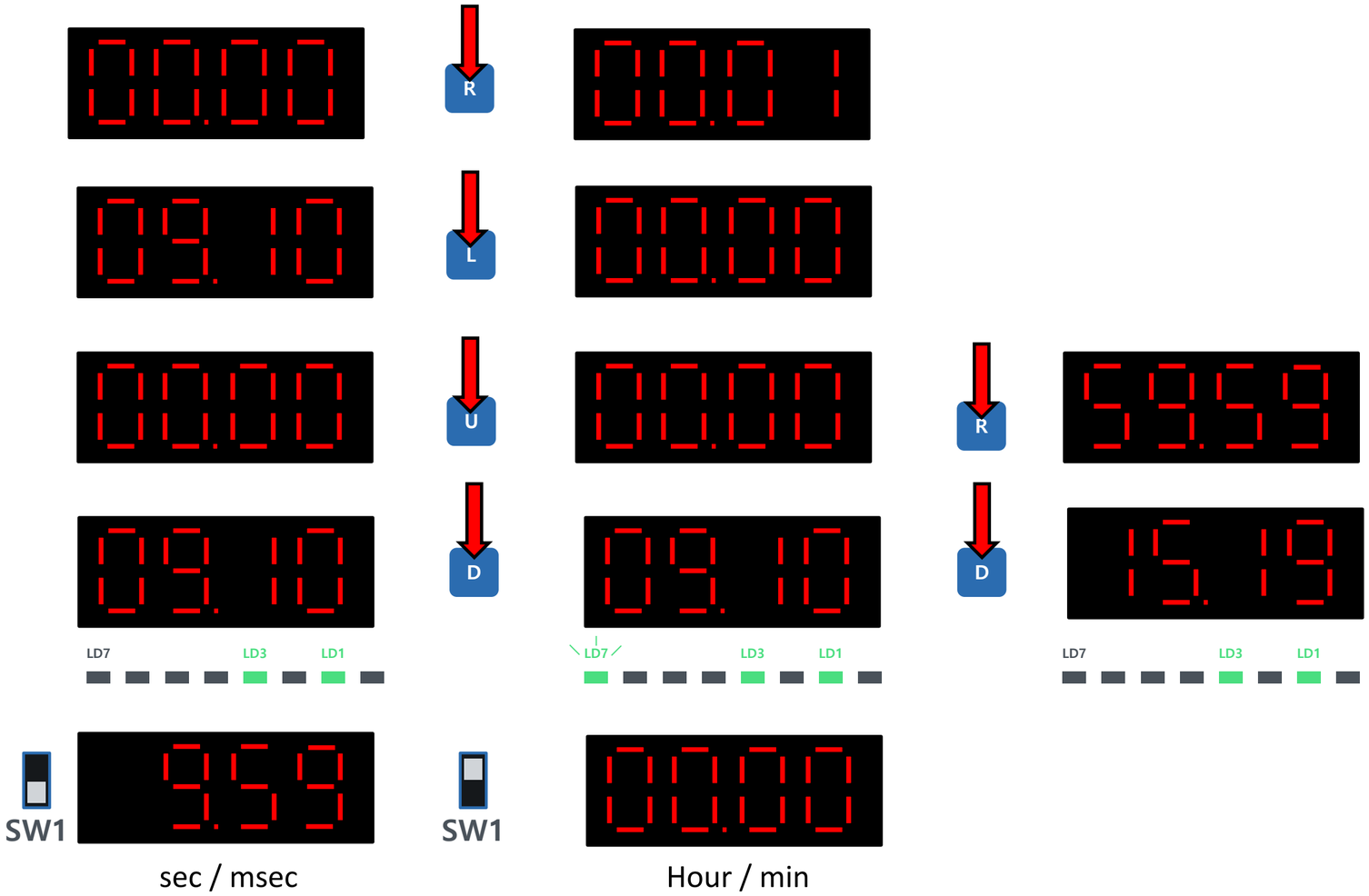
● 03-2 스톱워치 · 기능 설명

스톱워치 조작법 — 버튼 · 스위치별 동작

STOPWATCH SW4 = 0 · SW0 = 1 → LD1



#	입력	동작
①	**BTNR**	시작 / 정지 토글
②	**BTNL**	초기화 → 00:00:00.00 (정지 중)
③	**BTNU**	Up ↔ Down 방향 전환 (정지 중)
④	**BTND**	LAP : 시간 저장 · 표시 고정 / 다시 → 실시간
	SW1	화면 0 : 초.1/100초 · 1 : 시.분
	LED	LD1 스톱워치 · LD7 LAP · LD3 / LD2 화면



03-2 스톱워치 · 시뮬레이션

스톱워치 시뮬레이션 시나리오

요약 **R · L · U 기본 동작 3 개 + D 버튼 LAP 2 개 — 버튼마다 한 시나리오**

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과
1	**R 정지 · 재실행**	RUN 중 R → 정지 → R 재입력	`w_sw_run_stop · o_sw_msec`	**정지 중 값 유지, 재실행 후 증가**
2	**L 정지 중 초기화**	R 로 정지 → L	`w_sw_clear · o_sw_msec`	**Clear 펄스 → 16 → 0**
3	**U 감소 방향 전환**	정지 중 U → R 로 실행	`w_sw_down · o_sw_msec`	**Down 1 → 5 · 4 · … · 0 감소**
4	**D LAP 저장 · 표시 유지**	측정 중 D	`o_sw_lap · save_msec`	**내부 카운터 증가, 표시 21 유지**
5	**D LAP 해제 · 현재 값**	LAP 중 D 재입력	`save_msec · w_disp_msec`	**저장값 11 유지, 표시 17 (현재)**

규칙 실행 중 L · U 입력 무시 (STOP 에서만 동작) · D 는 화면에 보낼 값만 바꾸고 내부 측정은 계속
담당 이종혁 — LAP 구현 및 버튼 동작 검증

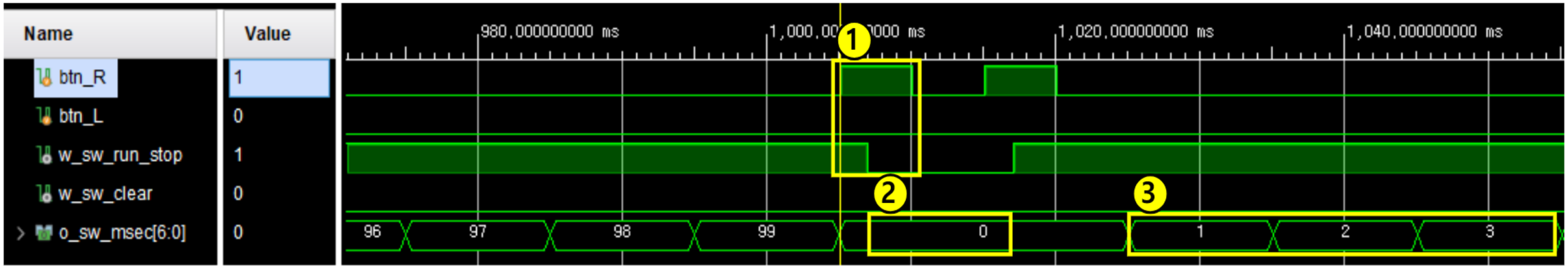
Note. 시나리오 3 의 99 → 5 구간은 감소를 짧게 보이기 위한 검증용 프리로드

03-2 스톱워치 · 시뮬레이션

시나리오 1 : R 버튼 — 정지 · 재실행

결과 **정지 중 값 유지, 재실행 후 카운터 증가**

시나리오 : R 입력으로 정지한 뒤 다시 R 입력으로 실행



01 첫 R 입력 후 실행 신호 1에서 0으로 전환

02 정지 구간에서 카운터 값 0 유지

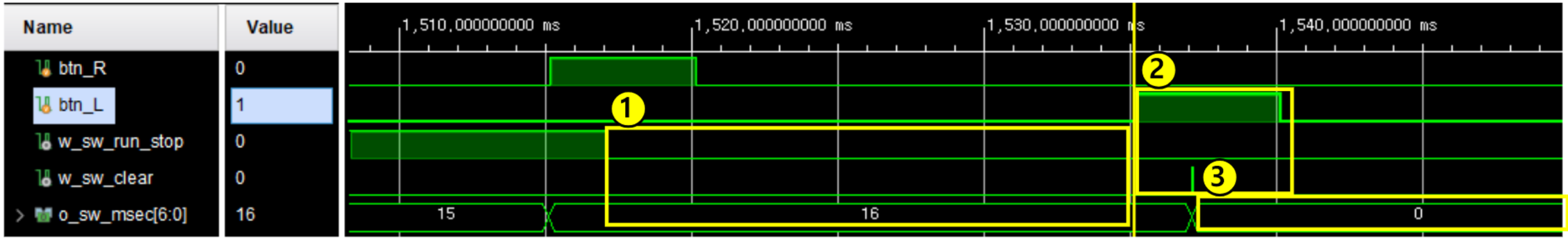
03 다시 R 입력 후 실행 신호 1, 카운터 1·2·3 증가

Figure 1. 스톱워치 R 버튼 정지 · 재실행 — Vivado xsim 파형

시나리오 2 : L 버튼 — 정지 중 초기화

결과 **Clear 신호 발생 후 시간 값 16 → 0 초기화**

시나리오 : R 로 정지한 뒤 L 입력으로 시간 초기화 확인



01 R 입력 후 정지, 카운터 값 16 유지

02 L 입력 후 Clear 펄스 발생

03 카운터 값 16에서 0으로 초기화

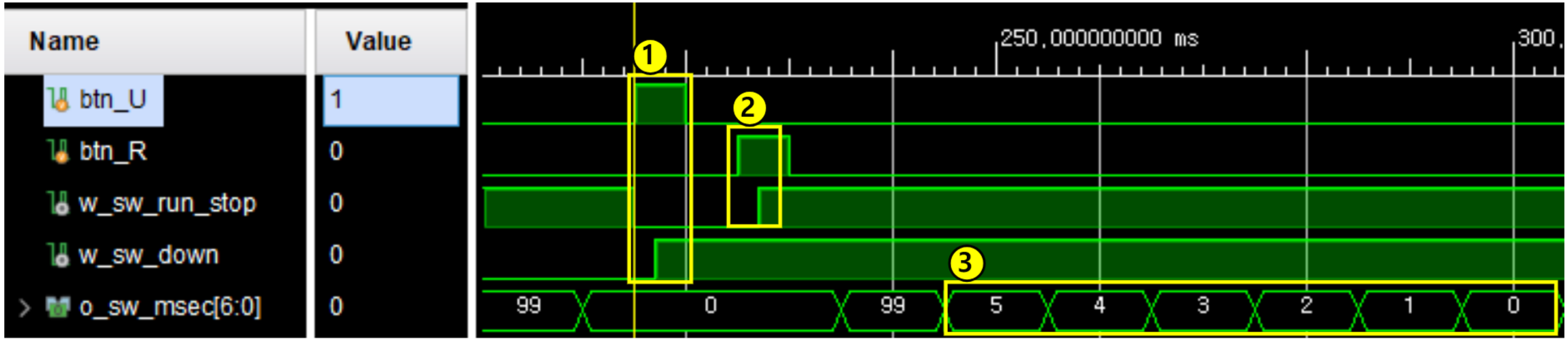
Figure 2. 스톱워치 L 버튼 정지 중 초기화 — Vivado xsim 파형

03-2 스톱워치 · 시뮬레이션

시나리오 3 : U 버튼 — 감소 방향 전환

결과 **Down 신호 활성화 후 카운터 감소**

시나리오 : 정지 중 U 입력 후 R 로 실행하여 감소 확인



01 정지 중 U 입력 후 Down 신호 0에서 1로 전환

02 R 입력 후 실행 신호 0에서 1로 전환

03 검증용 값 5 설정 후 5·4·3·2·1·0 감소

Figure 3. 스톱워치 U 버튼 감소 방향 전환 (99 → 5 는 검증용 프리로드) — Vivado xsim 파형

LAP 기능 코드 설계 — 저장 · 출력 선택

요약 **D 입력마다 LAP 토글, 켤 때만 저장 · 출력은 LAP 상태로 선택**

① 상태 전환과 시간 저장

```
if (i_sw_btn_D) begin
    lap_enable <= ~lap_enable;

    if (!lap_enable) begin
        save_msec <= i_cnt_msec;
        save_sec  <= i_cnt_sec;
        save_min  <= i_cnt_min;
        save_hour <= i_cnt_hour;
    end
end
```

② 출력값 선택

```
assign o_sw_msec =
    lap_enable ?
    save_msec : i_cnt_msec;
```

LAP 1: 저장값 출력
LAP 0: 현재 카운터 값 출력

초·분·시도 같은 방식으로 선택

현재 카운터는 이 선택과 별개로 계속 동작

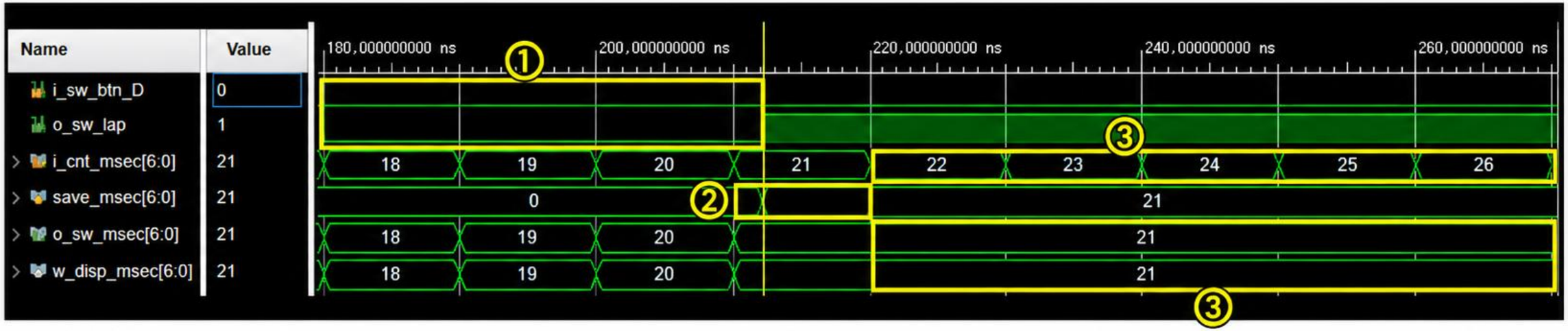
LAP을 켤 때만 저장하고, 해제할 때는 저장값 유지

Reset 또는 Clear 입력 시 LAP 상태와 저장값 초기화

시나리오 4 : LAP — 시간 저장 · 표시 유지

결과 **내부 카운터는 증가, 표시값은 21 유지**

시나리오 : 측정 중 D 버튼 입력 후 시간 저장과 표시 유지 확인



01 버튼 입력 후 LAP 상태 0에서 1로 전환

02 현재 카운터 값 21 저장

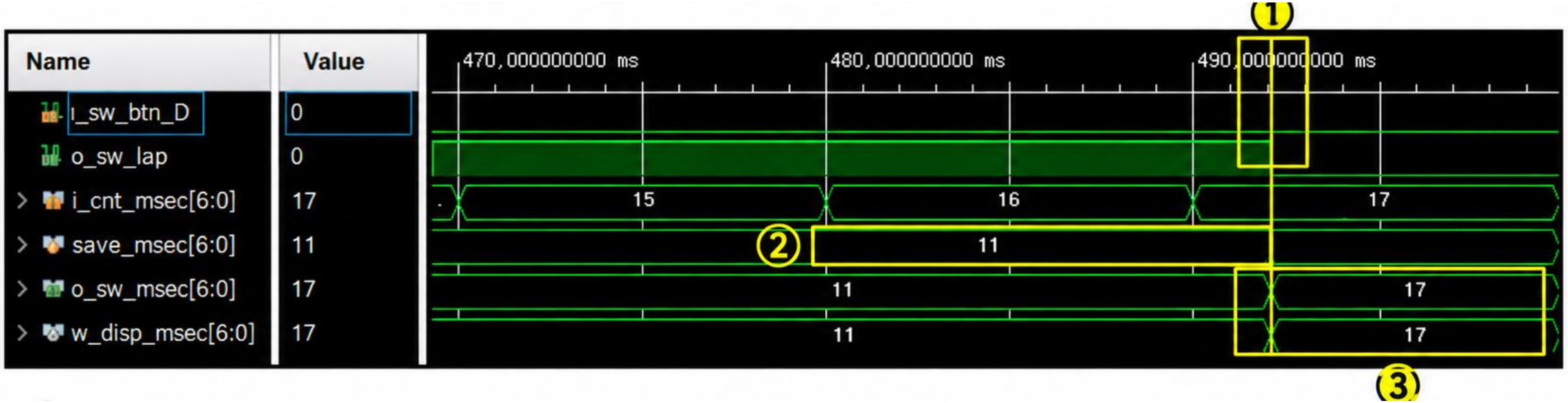
03 내부 값은 22~26 증가, 출력·표시값은 21 유지

Figure 4. 스톱워치 LAP 시간 저장 · 표시 유지 — Vivado xsim 파형

시나리오 5 : LAP — 해제 · 현재 값 표시

결과 **저장값 11 대신 현재 카운터 값 17 표시**

시나리오 : LAP 상태에서 D 버튼 재입력 후 현재 값 복귀 확인



01 버튼 입력 후 LAP 상태 1에서 0으로 전환

02 저장값 11은 해제 후에도 유지

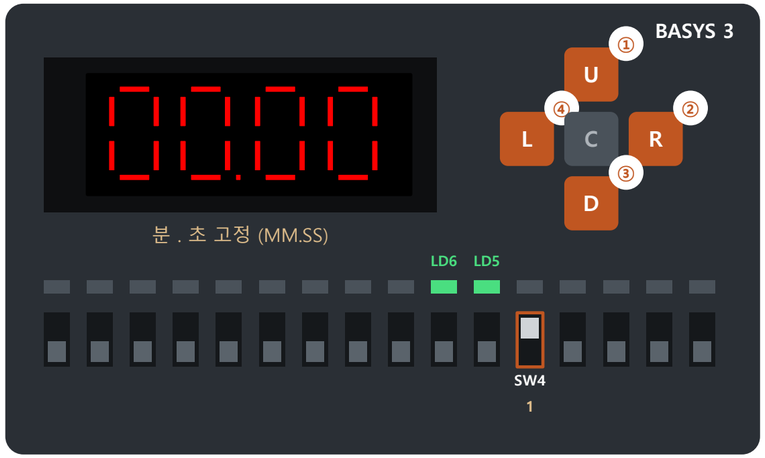
03 출력·표시값이 현재 카운터 값 17로 변경

Figure 5. 스톱워치 LAP 해제 · 현재 값 복귀 — Vivado xsim 파형

● 03-3 타이머 · 기능 설명

타이머 조작법 — 버튼 · 스위치별 동작

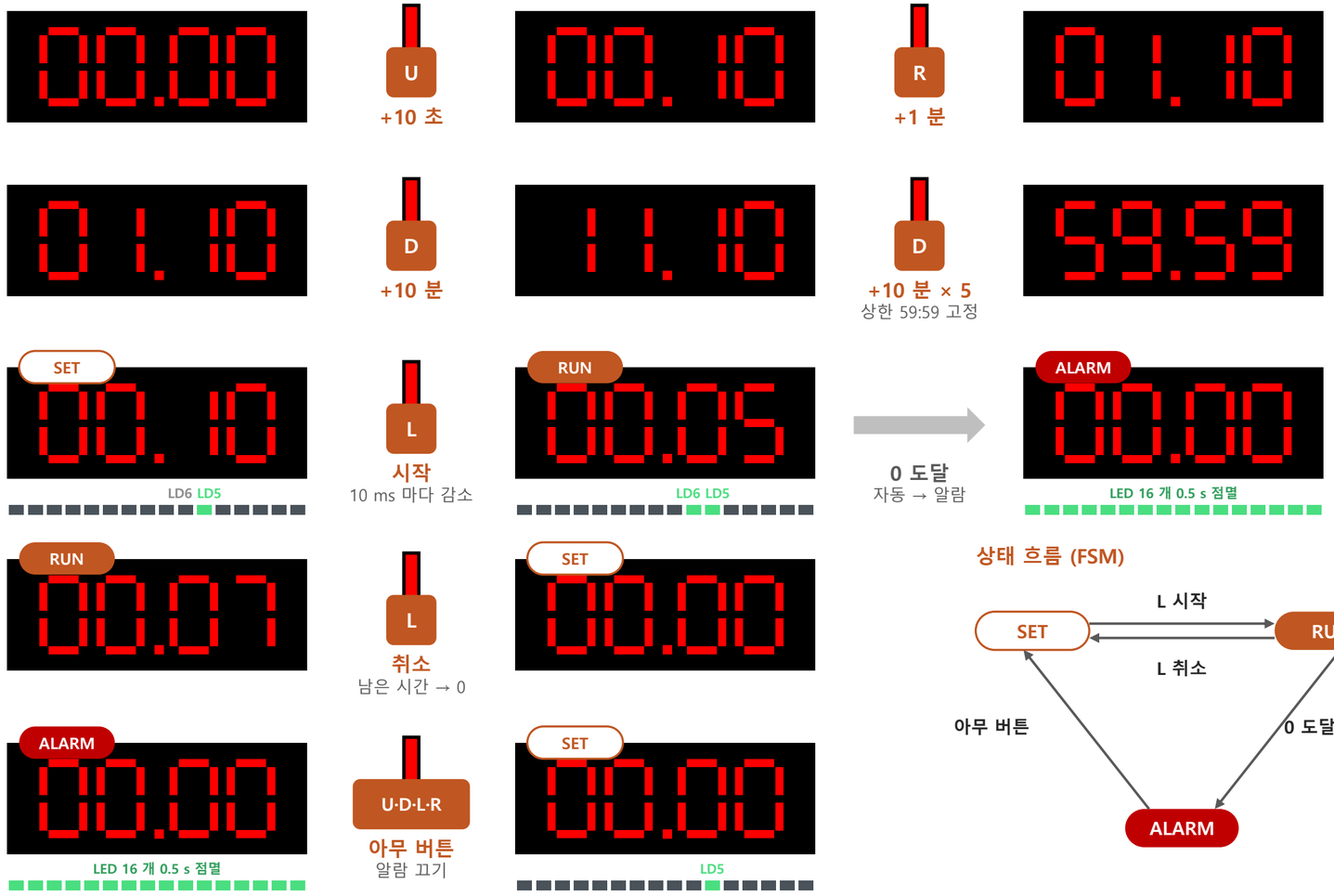
TIMER SW4 = 1 (우선) → LD5 · 진행 중 LD6



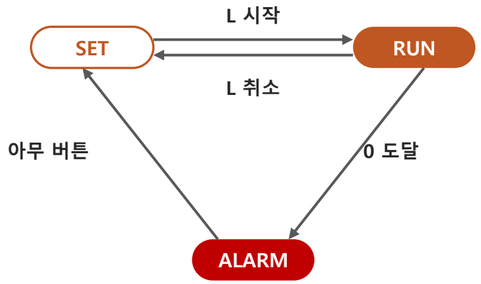
#	입력	동작
①	BTNU	+ 10 초
②	BTNR	+ 1 분
③	BTND	+ 10 분 (상한 59 : 59)
④	BTNL	시작 (0 이면 무시) · 진행 중 취소 → 0
	ALARM	0 도달 → LED 16 개 0.5 s 점멸
	끄기	알람 중 아무 버튼 → SET (값 유지)

화면 : 분 . 초 고정 (MM.SS)

SET 설정 RUN 감소 중 ALARM 알람



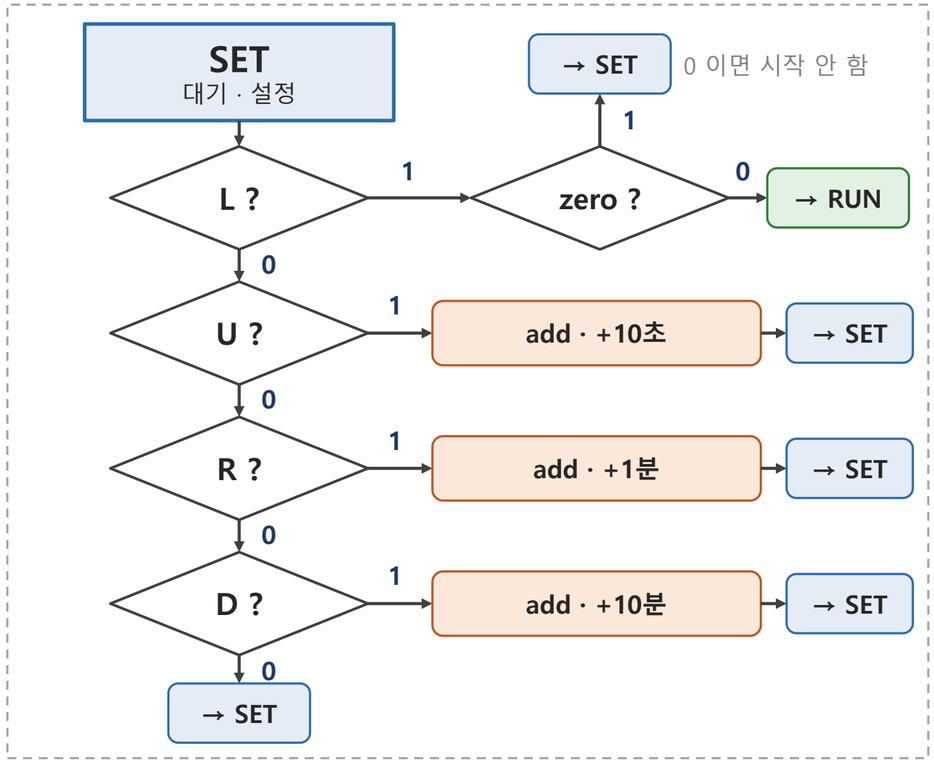
상태 흐름 (FSM)

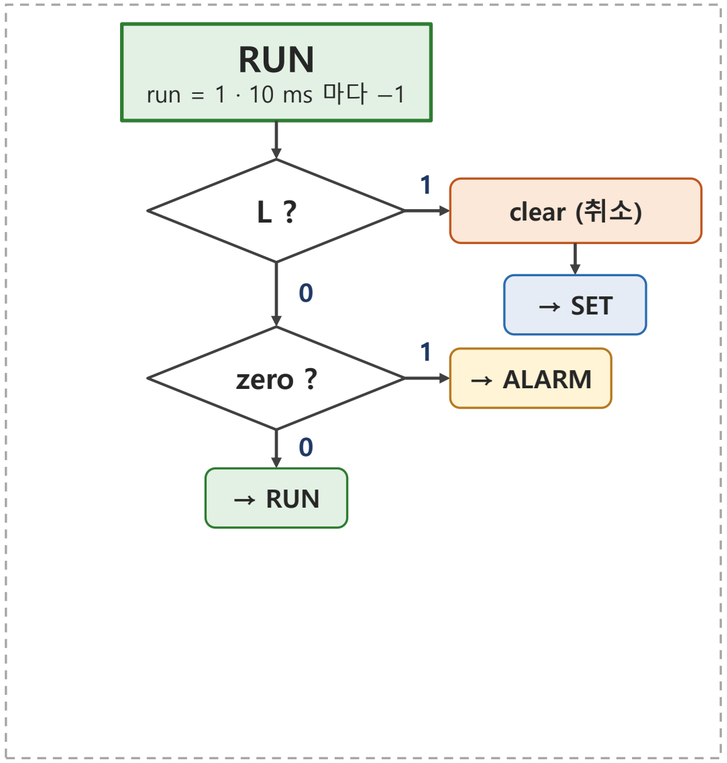


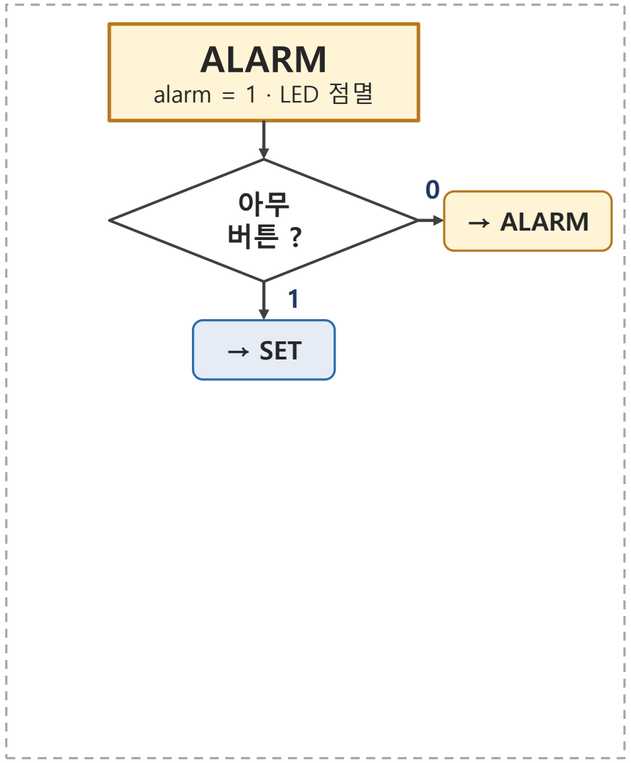
03-3 타이머 · 기능 설명

타이머 FSM — ASM 차트 (SET · RUN · ALARM)

요약 **SET 에서 더하고 L 로 시작 → RUN 에서 줄다가 0 이면 ALARM → 아무 버튼으로 끔**







□ **상태** 안의 출력은 그 상태 내내 1 ◇ **조건** 버튼 · zero ▭ **주황** 조건부 출력 (그 클럭에만 1)

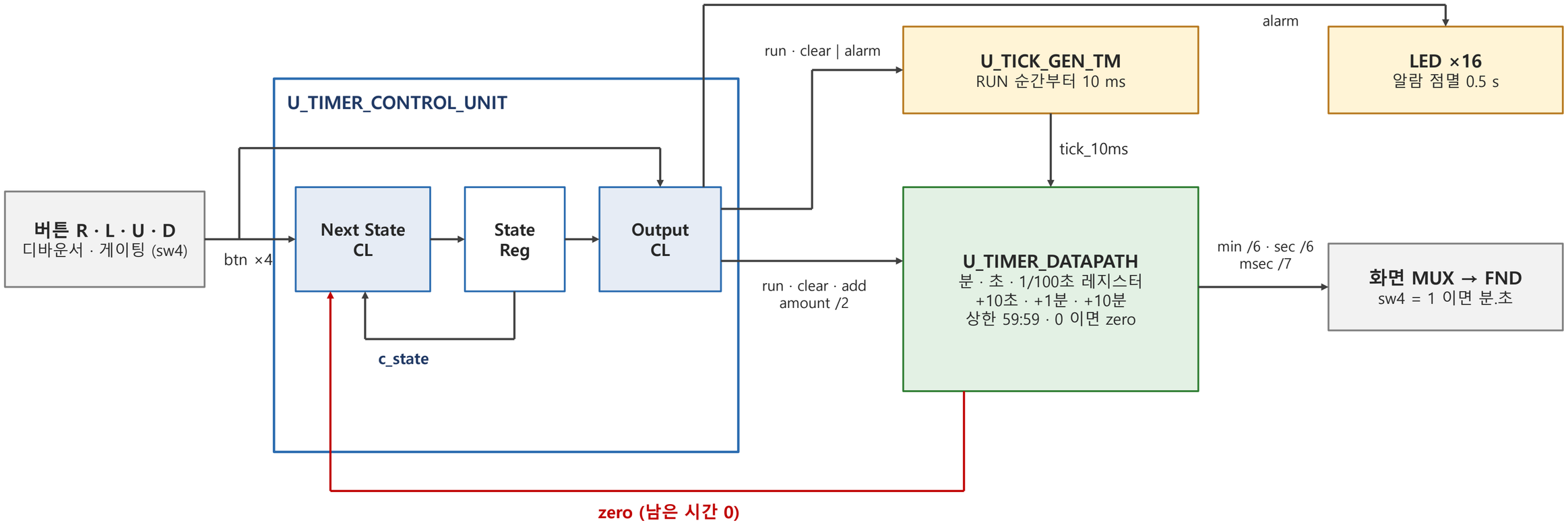
zero = 데이터패스의 남은 시간이 0 **add** = 데이터패스에 더하기 (상한 59:59)

Figure. 타이머 ASM 차트 — timer_control_unit.sv 의 조건 · 출력과 같음

03-3 타이머 · 기능 설명

타이머 구조 — 컨트롤 유닛 · tick 생성기 · 데이터패스

요약 **컨트롤 유닛이 run · clear · add 를 내고, 데이터패스가 남은 시간을 세다 0 이면 zero 를 돌려줌**



출력 run · alarm 은 상태로 정해짐 (무어) · add · clear · amount 는 버튼 누른 클럭에만 (밀리)

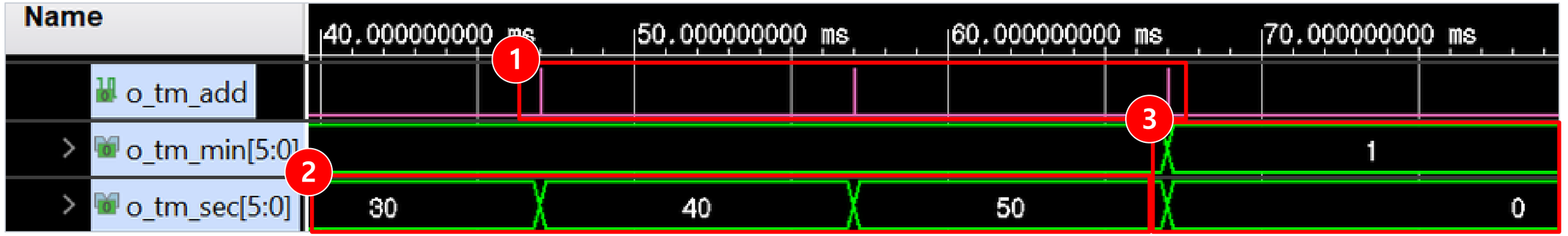
되먹임 State Reg 의 c_state 를 Next State CL 로 되돌려 다음 상태를 정함

Figure. 타이머 블록도 — 모듈 · 신호 이름은 소스(verification_v2/source_file) 그대로

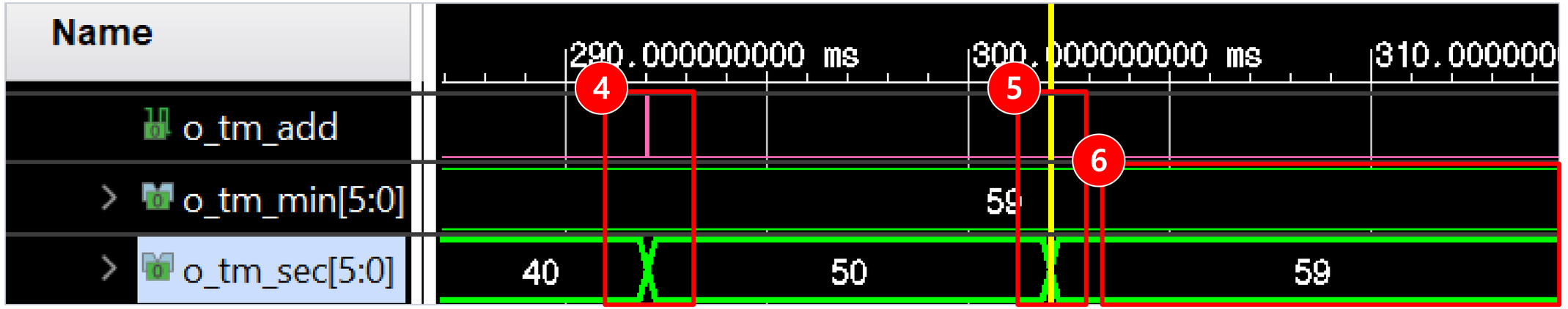
03-3 타이머 · 시뮬레이션

시나리오 1 · 2 : 더하기 · 자리 올림 · 59:59 상한

결과 **00:50 + 10 초 = 01:00 (60 초 아님) · 59:50 + 10 초 → 59:59 에서 멈춤 (60:00 아님)**



① U 4 · 5 · 6번째 add 펄스 ② 초 30 → 40 → 50 ③ **6번째: 분 1 · 초 0**

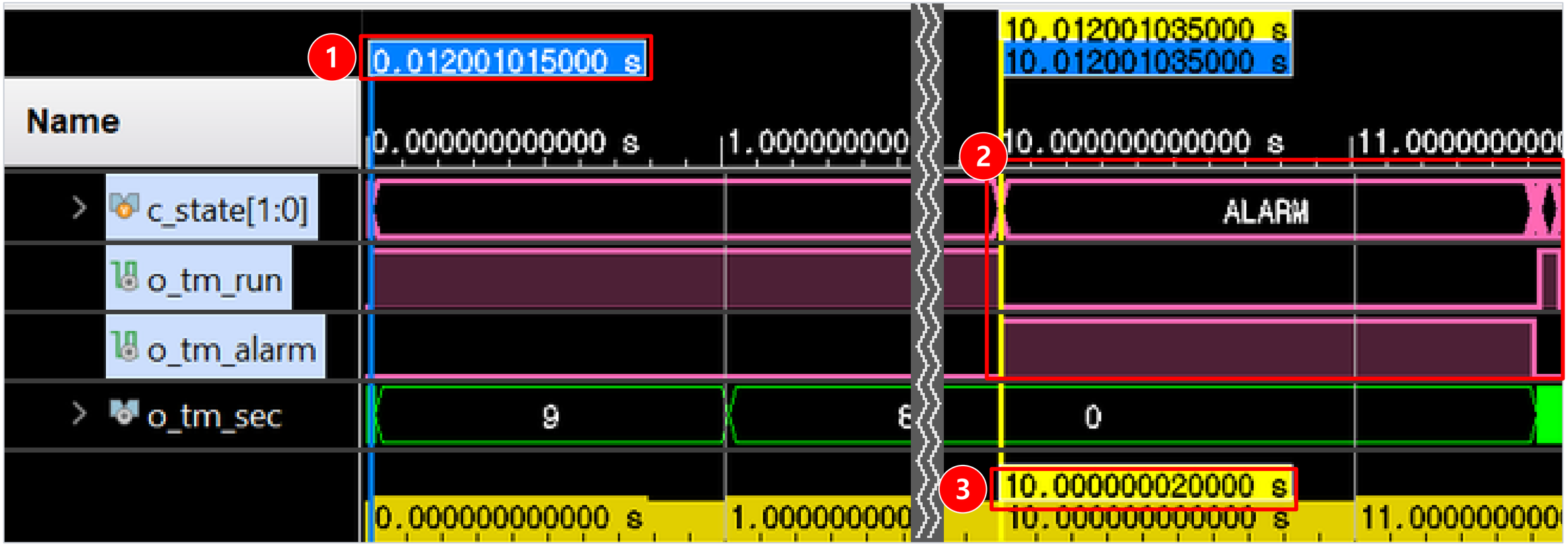


④ U: 59:40 → 59:50 ⑤ **U: 59:50 → 59:59** ⑥ 59:59 유지 (60:00 아님)

Figure 6. 타이머 더하기 · 59:59 상한 (위 : 더하기, 아래 : 상한) — Vivado xsim 파형

시나리오 3-1 : 10 초 카운트다운 → 알람

결과 **00:10 설정 → 시작부터 알람까지 10.000000020 s (= 10 s + 2 클럭)**



① RUN 시작 커서 0.012 s ② **RUN → ALARM, alarm 1** ③ **간격 10.000000020 s**

Figure 7. 타이머 10 초 카운트다운 → 알람 (가운데 ≋ = 1 ~ 9 s 생략, 시뮬: 배치 실행) — Vivado xsim 파형

04

트러블 슈팅

분석 순서
① 현상 → ② 원인 → ③ 해결 → ④ 적용 결과 (Before / After)

1. 10 초 타이머가 9.998 초에 울림

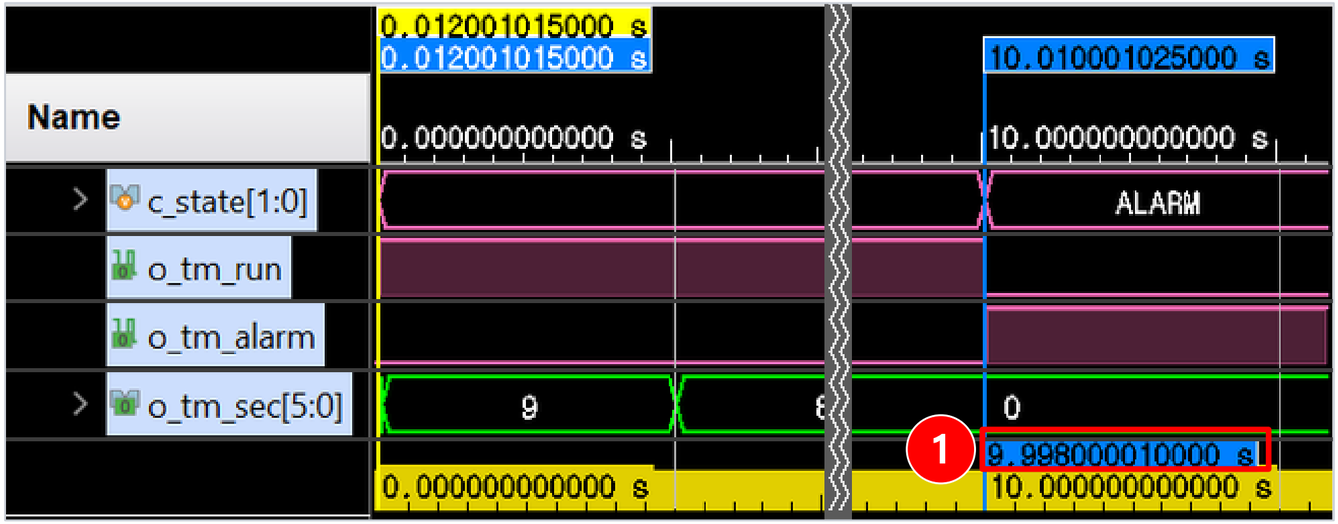
100 Hz tick 공유 → 기능별 tick 분리

2. 한 번 눌렀는데 두 번 인식

디바운서 샘플 1 MHz → 4 kHz

트러블 슈팅 1 — 10 초 타이머가 9.998 초에 울림

원인 **원인 : 100 Hz tick 1 개를 시계 · 스톱워치 · 타이머가 공유 → 시작 시점 따라 최대 10 ms 일찍**

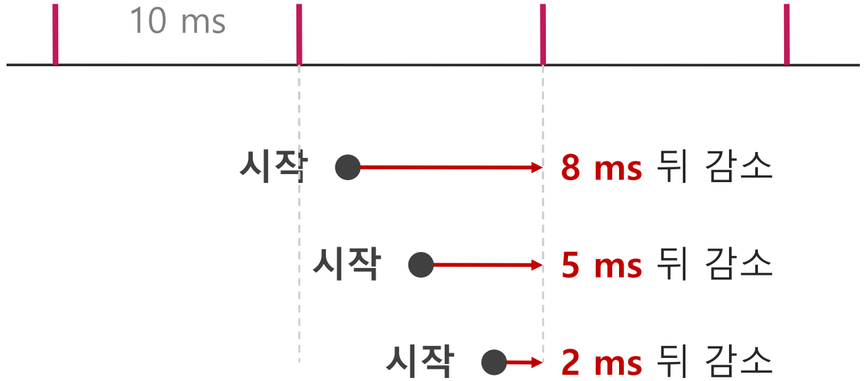


① 현상 RUN 시작 → ALARM 간격 **9.998000010 s**

10/17 캡처 · 가운데 ≈ = 1~9 s 생략

② 원인

공유 tick (리셋부터 항상 10 ms 박자)



시계: 시작이 없어 상관없음

스톱워치 · 타이머: 누른 순간이 0 이어야 함

측정 tb_tm_2 P1 9,998 ms · 시작 시각을 3 ms 씩 바꿔 돌린 TB 에서 첫 감소 8 / 5 / 2 ms

트러블 슈팅 1 — 해결 : tick 구조 4 종 비교

해결 **④ 를 선택: 4 kHz 는 같이 쓰고, 100 Hz 만 기능마다 따로 → 매번 정확히 10 ms**

판	구조	LUT	FF	WNS	시작 → 첫 tick
①	수업 방식: 디바운서마다 분주 + 데이터패스마다 tick	415	292	+3.28	매번 10 ms
②	기존 팀 판: 4 kHz 공유 + 100 Hz 1개 셋이 공유	321	186	+2.42	**최대 10 ms 일찍**
③	참고: ② + 4 kHz 를 enable 로	311	172	+1.50	② 와 같음
④	**4 kHz 공유 + 100 Hz 기능마다 따로 (RUN 순간부터)**	**384**	**228**	**+3.44**	**매번 정확히 10 ms**

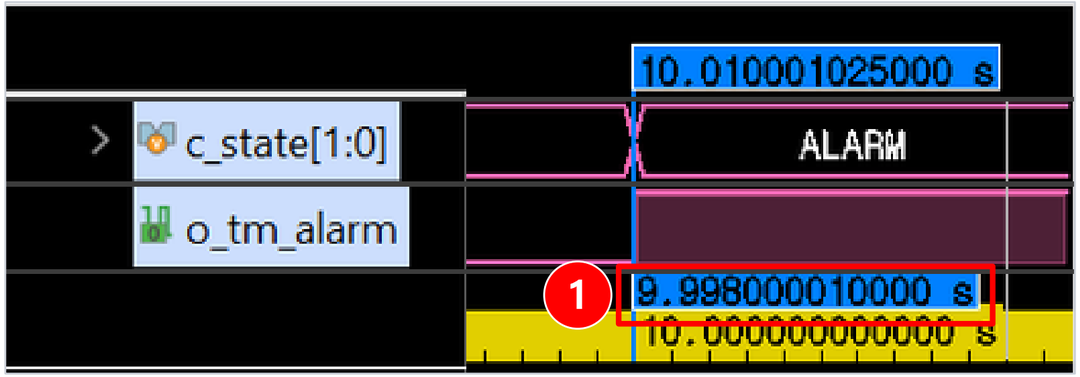
선택 이유 100 Hz tick 은 누른 순간부터 세야 해서 기능마다 따로 두고, 4 kHz 는 시작 시점과 상관없어서 하나를 같이 씀

대가 ② 보다 FF 42 개 증가 (tick 생성기 2 개 × 21 비트) · ① 수업 방식보다는 LUT 31 개 · FF 64 개 적음

Vivado 2022.2 · 구현 후 · 네 판 모두 최종 코드 · ①③ 은 비교용으로 만든 판 (측정: 배치 실행)

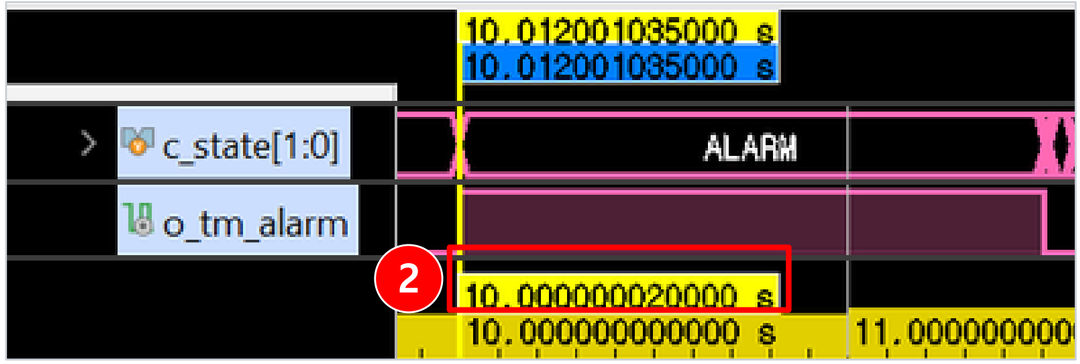
트러블 슈팅 1 — 적용 결과

결과 **9.998000010 s → 10.000000020 s (10 s + 2 클럭 고정 지연) · 잔여 2 건도 PASS**



Before 간격 **9.998000010 s**

시작 시점에 따라 **최대 10 ms 일찍** (이 캡처는 2 ms)



After 간격 **10.000000020 s**

= 10 s + **20 ns** = 2클럭 고정 (매번 같음)

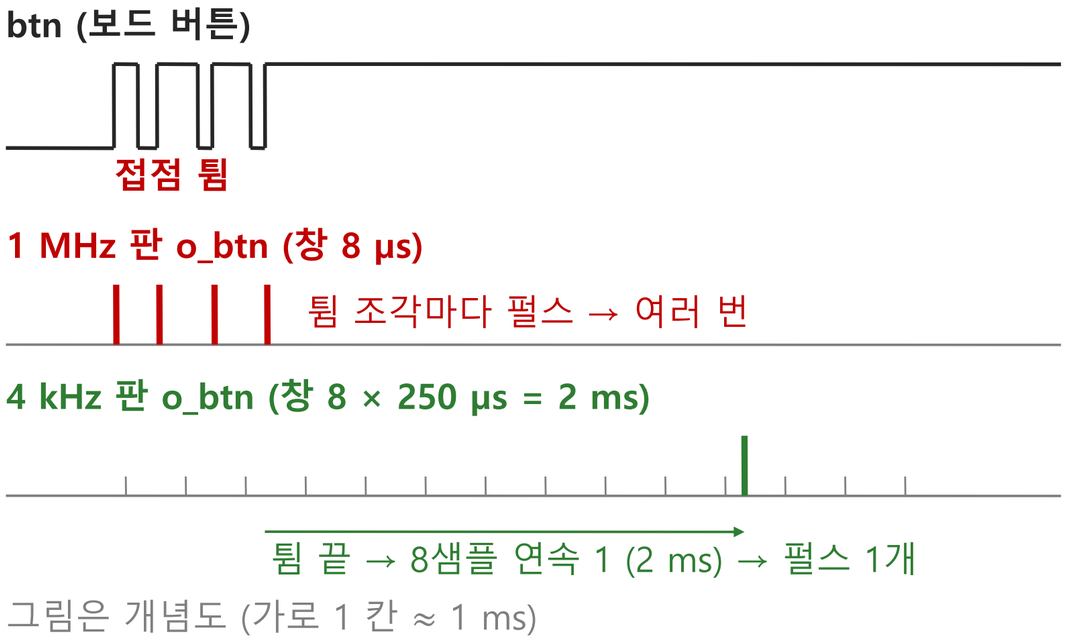
```
# 잔여 2건 — 같은 TB 를 고치기 전 / 고친 뒤 소스로 (콘솔 요약)
[before] 알람 뒤 재시작 → 첫 tick = 9999980 ns   FAIL
[before] tick 클럭에 STOP → msec = 4 (10 ms 잃음)  FAIL
[after ] 알람 뒤 재시작 → 첫 tick = 10000000 ns
[after ] tick 클럭에 STOP → msec = 5   TOTAL MISMATCH = 0
```

고친 것: 알람 때도 tick 카운터를 0 으로 · 스톱워치 데이터패스의 중복 조건 삭제

트러블 슈팅 2 — 한 번 눌렀는데 두 번 인식

① 현상 ② 원인 ③ 해결 ④ 적용 결과

결과 **디바운서 샘플 1 MHz → 4 kHz : 같은 튐 입력 20 번 → 펄스 158 개 → 20 개**



그림은 개념도 (가로 1 칸 ≈ 1 ms)

③ 해결 — 4 kHz 하나로 두 가지

디바운서

CLK_FRQ 4000 → 250 µs 마다 샘플, 8번 연속 1 = 2 ms 눌려야 인정

FND 스캔 (같은 4 kHz)

8칸 × 250 µs = 한 바퀴 2 ms (500 Hz)

Basys-3 설명서 권장 한 바퀴 1 ~ 16 ms 안 → 깜빡임 없음

→ 분주기 1개를 디바운서 4개 + FND 가 공유

④ 결과	Before 1 MHz	After 4 kHz
시뮬: 같은 튀는 입력 20번 → o_btn 개수	**158개 (20번 모두 틀림)**	**20개 (모두 1번)**
대가	반응 8 µs	반응 1.75 ~ 2 ms (못 느낌)

튐 모델: 누를 때 · 뗄 때 20 ~ 400 µs 조각 4 ~ 11개 (실제보다 심함) · Icarus 실행

결론 · 고찰

결론 **목표 기능 전부 Top TB PASS · 보드 확인 — 고찰 : 남은 경고 · 입력 처리 한계 → 다음 목표**

기능	목표	Top TB	보드
스톱워치	시작 · 정지 · 지우기 · 업다운 · 랩 · 자정	**53 PASS**	**확인**
시계	12:00 시작 · 설정 · 점멸 · 12/24 표시	**264 PASS**	**확인**
타이머	더하기 · 59:59 상한 · 10 s · 알람	**73 PASS**	**확인**
공통	디바운서 · FND 스캔 · 동시 입력 무시	**57 PASS**	**확인**

구현 LUT 384 (1.8%) · FF 228 (0.5%) · WNS +3.440 ns @100 MHz · 래치 0
1/100초 = 10.000 ms · 10초 타이머 = 10 s + 20 ns

결론 FSM(언제) + 데이터패스(얼마나) 분리로 세 기능을 같은 구조로 만들고, 버튼 → FND 전체 경로를 100 MHz 그대로 시뮬레이션해 목표 기능이 모두 사양대로 동작함을 확인

고찰 (아쉬운 점 → 다음 목표)

① 경고 35개 남음

4 kHz 펄스를 클럭으로 씀 (TIMING-17) → enable 방식이면 0 개 (비교용 판 확인) → 다음 프로젝트 적용

② 디바운서 한계

8번 연속 1 만 봄 → 누른 채 잠깐 끊기면 두 번 인식 가능 → 뗄 때도 8번 연속 0 보기 검토

③ 긴 경계

23:59:59 등은 초기값으로 확인 → 실제 24시간 흐름은 보드로만

06 시연 영상

시연 ① 스톱워치 — 보드 동작 (GIF 자동 반복)

요약 **1분 올림 · 랩 · 동작 중 L 무시 · 0 ↔ 23:59:59.99 를 Basys-3 에서 확인 — 시뮬레이션(16 ~ 21 장) 과 같은 장면**













보드에서 본 것 59.99 다음 분 +1 · 랩 중에도 안에서는 계속 셈 · 동작 중 L 무시 · 다운 0 → 23:59:59.99 · 업 23:59:59.99 → 00:00

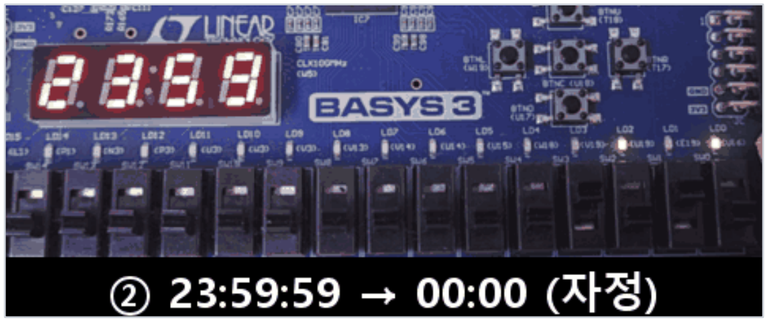
GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55) · 촬영 정보윤

06 시연 영상

시연 ② 시계 — 보드 동작 (GIF 자동 반복)

요약 **자정 넘김 · 12시간 표시 · 설정 점멸 · 자리별 ± 를 Basys-3 에서 확인 — 시뮬레이션(9 ~ 13 장) 과 같은 장면**











보드에서 본 것 리셋 23:59:50 → 10 초 뒤 자정 · 12시간 표시에서 0시 = 12 · 설정 중 커서 자리 깜빡 · 그 자리만 ±

GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55) · 촬영 정보윤

06 시연 영상

시연 ③ 타이머 — 보드 동작 (GIF 자동 반복)

요약 **0 에서 시작 안 함 · 1분 올림 · 59:59 상한 · 취소 · 다른 화면에서도 알람 을 Basys-3 에서 확인 — 시뮬레이션(25 · 26 장) 과 같은 장면**















보드에서 본 것 0 에서 시작 안 함 · 60초 = 1분 · 최대 59:59 · 동작 중 L = 취소 · 화면을 바꿔도 계속 줄고 0 이면 어느 화면이든 알람 · 버튼은 알람만 끔

GIF (슬라이드쇼에서 자동 반복) · 시연용 빌드 = 리셋값만 경계 바로 앞 (시계 23:59:50 · 스톱워치 00:00:55) · 촬영 정보윤

Q & A

감사합니다

부록 A — 검증 전체 : TB 13 개 · 검사 447 개 PASS

요약 **Top 을 넣은 TB · 100 MHz 그대로 · 콘솔 [SUMMARY] 자동 판정 · 최신 소스로 다시 실행**

폴더	TB	확인하는 것	검사	결과	시뮬 시간
top	tb_top_1	디바운서: 누름 시간 · 바운스	24	PASS	2,205 ms
top	tb_top_2	FND 스캔 · 자리별 출력 · 점	22	PASS	2,180 ms
top	tb_top_3	버튼 동시 입력: 세 기능 모두 무시 (10/18)	11	PASS	302 ms
스톱워치	tb_sw_1	시작 · 정지 · 지우기 · 동작 중 무시 · 동시 R+L	23	PASS	2,290 ms
스톱워치	tb_sw_2	올림 · 자정 순환 · 다운 빌림	15	PASS	485 ms
스톱워치	tb_sw_3	랩 고정 · 해제 · 지우기	15	PASS	1,230 ms
타이머	tb_tm_1	더하기 · 올림 · 59:59 상한	21	PASS	455 ms
타이머	tb_tm_2	실제 10 초 · 알람 · 빌림	41	PASS	11,597 ms
타이머	tb_tm_3	취소 · 백그라운드 · 화면 우선순위	11	PASS	547 ms
시계	tb_wt_1	설정 UI · 커서 · 경계	38	PASS	870 ms
시계	tb_wt_2	1초 정확도 · 자정 올림	13	PASS	2,245 ms
시계	tb_wt_3	12/24 표시 · 점멸 · LED	118	PASS	6,050 ms
시계	tb_wt_4	설정 중 R · L · U · D 동시 누름 16가지	95	PASS	480 ms

최종 코드(동시 입력 무시 포함)로 13 개 전부 다시 실행 · xsim 배치 (, 10/18)

부록 질문 대비

부록 B — 버튼 두 개를 같은 순간에 누르면? → 모두 무시

요약 **같은 클럭에 버튼 펄스 2개 이상 → 그 클럭 입력 모두 버림 (세 기능 같은 규칙, 10/18)**

화면 · 상태	같은 클럭에 동시 입력	전 (기능마다 우선순위)	지금 (10/18)
스톱워치 STOP	R + L	RUN 시작 + 0 으로 지움	**무시 · STOP · 값 유지**
시계 설정	L + R	L 우선 (커서 왼쪽)	**무시 · 커서 그대로**
시계 설정	U + D	무시 (10/18 오전 수정)	**무시 · 값 그대로**
타이머 SET	U + R	U 만 (+10초)	**무시 · 00:00 그대로**
타이머 SET	L + D	L 만 (시작)	**무시 · 시작 안 함**
버튼 하나만	L · U 등	보통 동작	**보통 동작 (그대로)**

어디서 Top 의 버튼 게이팅: w_btn_multi = 펄스 두 개 이상 → 12개 게이팅 AND 에 ~w_btn_multi (컨트롤 유닛 · 랩까지 한 번에)
확인 tb_top_3 (세 기능 동시 입력 · 검사 11개) · tb_sw_1 P6 · tb_wt_4 (시계 16가지) — 고치기 전 소스로 돌리면 tb_top_3 FAIL 7개

같은 클럭이 되는 조건: 디바운서 4 kHz(250 µs) 샘플 → 두 버튼이 250 µs 안에 같이 안정될 때만

부록 C — 합성 · 구현 전략 비교 (PPA) → 기본값 유지

요약 **자원 1.8% · WNS +3.44 ns · 0.106 W 로 셋 다 여유 → 제약이 없어 기본값 유지 (Area 는 LUT −13.5% ↔ WNS −0.84 ns)**

전략 (Vivado 2022.2)	LUT	FF	WNS @ 100 MHz	전력 (추정)
기본값 (Vivado Defaults) — 채택	384	228	+3.440 ns	0.106 W
합성 Area (Flow_AreaOptimized_high)	332 (−13.5%)	227	+2.600 ns	0.115 W
합성 Area + 구현 Area_Explore	336	227	+2.011 ns	0.114 W
성능 (PerfOptimized_high + Performance_Explore)	384	228	+3.440 ns	0.106 W
전력 (Power_DefaultOpt, power_opt_design)	390	228	+3.733 ns	0.104 W

왜 기본값 전략은 빡빡한 제약 쪽을 고르는 것 — 이 설계는 칩의 1.8%, 타이밍 실패 경로 0, USB 전원이라 셋 다 제약이 없음. Area 를 고르면 얻는 것 없이 타이밍 여유만 줄고, Power 효과는 추정 오차 안 (전체 0.106 W 중 0.072 W 는 칩 누설)
네 판 모두 같은 경향 트러블 슈팅 1 표의 ① ~ ④ 에 같은 전략을 적용해도 Performance 는 전부 기본값과 동일, 합성 Area 는 LUT −7 / −28 / −26 / −52 에 WNS 전부 감소, Power 는 0 ~ −0.002 W

최종 코드 · 같은 xdc · xc7a35t · 구현 후 · 전력은 Vivado 추정(신뢰도 Low) · 측정은 배치 실행 (design_prototype/ppa_v3)

부록 타이머 검증 상세

타이머 시뮬레이션 시나리오

결과 **TB 3 개 · 검사 73 개를 $display 로 자동 판정 → 전부 PASS**

#	시나리오	입력 (Stimulus)	확인 신호	예상 결과	TB
1	**더하기 · 자리 올림**	SET 에서 U 6 번 (+10 초씩)	o_tm_add · o_tm_min/sec	**00:50 + 10 초 = 01:00**	TM1
2	**59:59 상한**	59:40 에서 U 2 번	o_tm_min · o_tm_sec	**59:50 → 59:59 유지 (60:00 X)**	TM1
3	**10 초 카운트다운 · 알람**	00:10 설정 → L 시작	c_state · o_tm_alarm · led	**알람까지 10 s + 20 ns · LED 0.5 s 점멸**	TM2
4	**다운카운트 빌림**	01:00.00 · 10:00.00 에서 tick	o_tm_min/sec/msec	**00:59.99 · 09:59.99**	TM2
5	**백그라운드 · 화면 우선순위**	RUN 중 sw4 1 → 0 / 알람 중 R	c_state · o_tm_alarm · led	**시계 화면에서도 알람 · R 은 알람만 끔**	TM3

```
# Vivado xsim 콘솔 — 팀 TB (SystemVerilog) 마지막 줄
[  455.019 ms] [SUMMARY] TM1 checks=21 fail=0 => PASS
[11597.002 ms] [SUMMARY] TM2 checks=41 fail=0 => PASS
[  547.009 ms] [SUMMARY] TM3 checks=11 fail=0 => PASS
# 같은 시나리오를 수업 Verilog 문법 (err 카운트) 으로 옮긴 TB
TM1 checks = 21, TOTAL MISMATCH = 0   (TM2 41 · TM3 11 도 0)
```

73 / 73

검사 PASS (TM1 21 · TM2 41 · TM3 11)

0

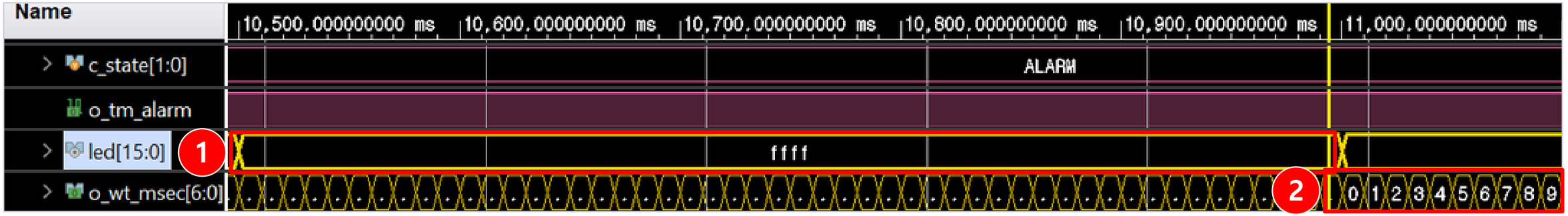
TOTAL MISMATCH (Verilog TB)

Note. 판정 예 — if (값 !== 기대값) err = err + 1; $display("FAIL ...") · 캡처는 10/17 소스, 결과 숫자는 10/18 최종 소스 기준 (둘 다 전부 PASS)

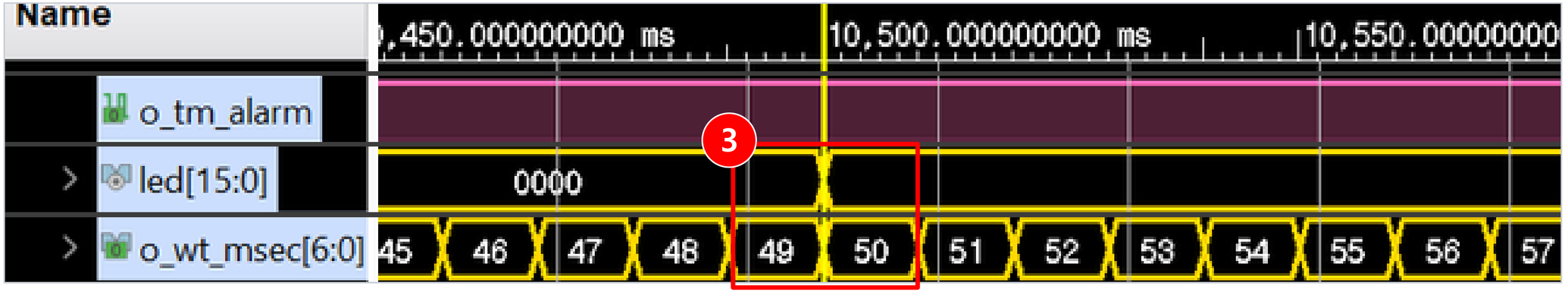
부록 타이머 검증 상세

시나리오 3-2 : 알람 LED 점멸

결과 **알람 중 LED 16 개가 0.5 s 켜짐 · 0.5 s 꺼짐 반복**



① led = ffff, 0.5 s 동안 켜짐 ② 시계 1/100초 00 → led 0000 (꺼짐)



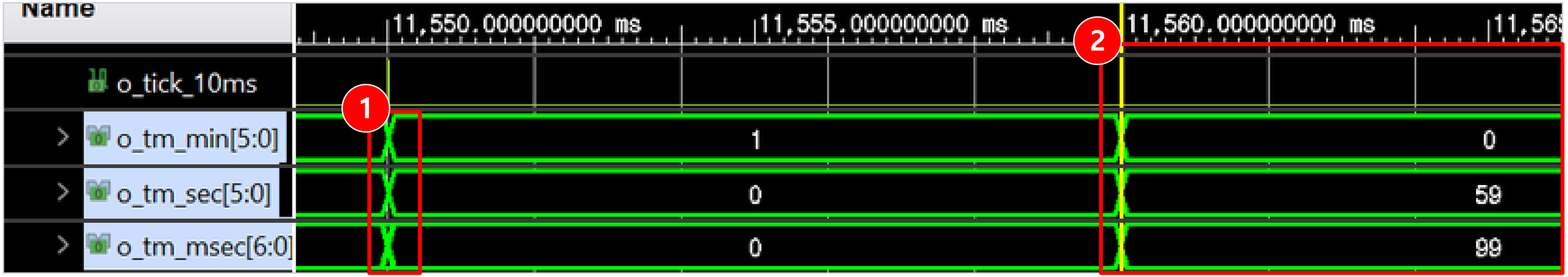
③ 확대: 시계 1/100초 49 → 50 인 클럭에 0000 → ffff

Figure 8. 타이머 알람 LED 점멸 (아래 : 49 → 50 확대) — Vivado xsim 파형

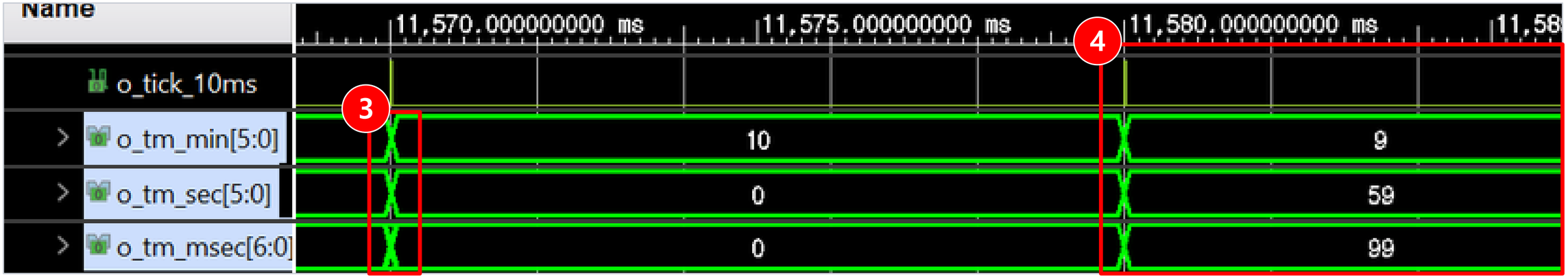
부록 타이머 검증 상세

시나리오 4 : 다운카운트 빌림

결과 **빌림 정상 — 01:00.00 → 00:59.99, 10:00.00 → 09:59.99**



① 01:00.00 넣음 ② **다음 tick → 00:59.99**



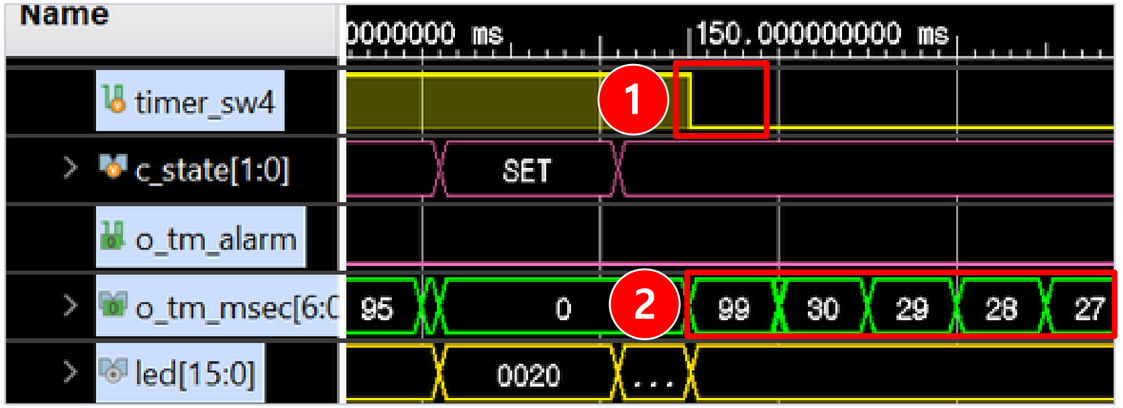
③ 10:00.00 넣음 ④ **다음 tick → 09:59.99**

Figure 9. 타이머 다운카운트 빌림 — Vivado xsim 파형

부록 타이머 검증 상세

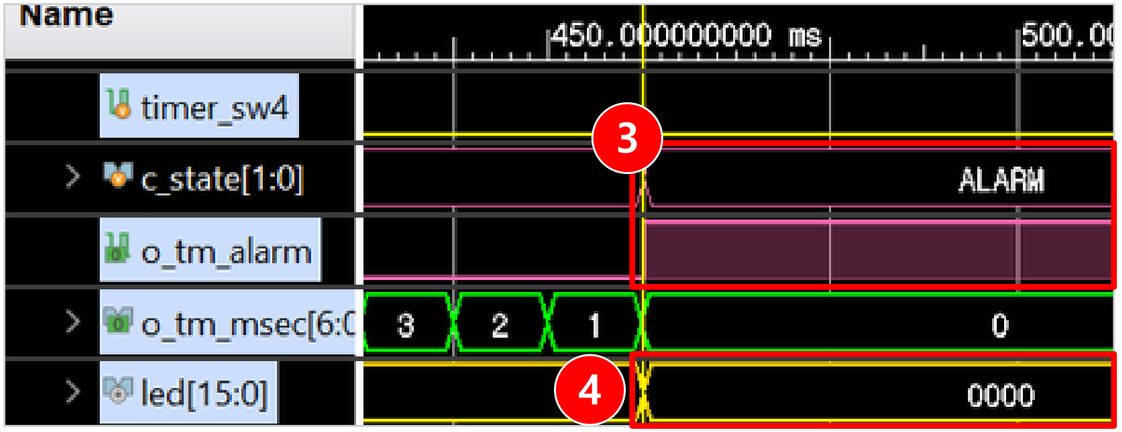
시나리오 5-1 : 백그라운드 동작

결과 **시계 화면으로 바꿔도 타이머는 계속 줄고, 0 이 되면 알람**



① timer_sw4 1 → 0 (시계 화면)

② 값은 계속 줄어듦 99 → 30 → 29 …



③ **RUN → ALARM, alarm 1**

④ LED 점멸 (이 순간은 꺼진 쪽 0000)

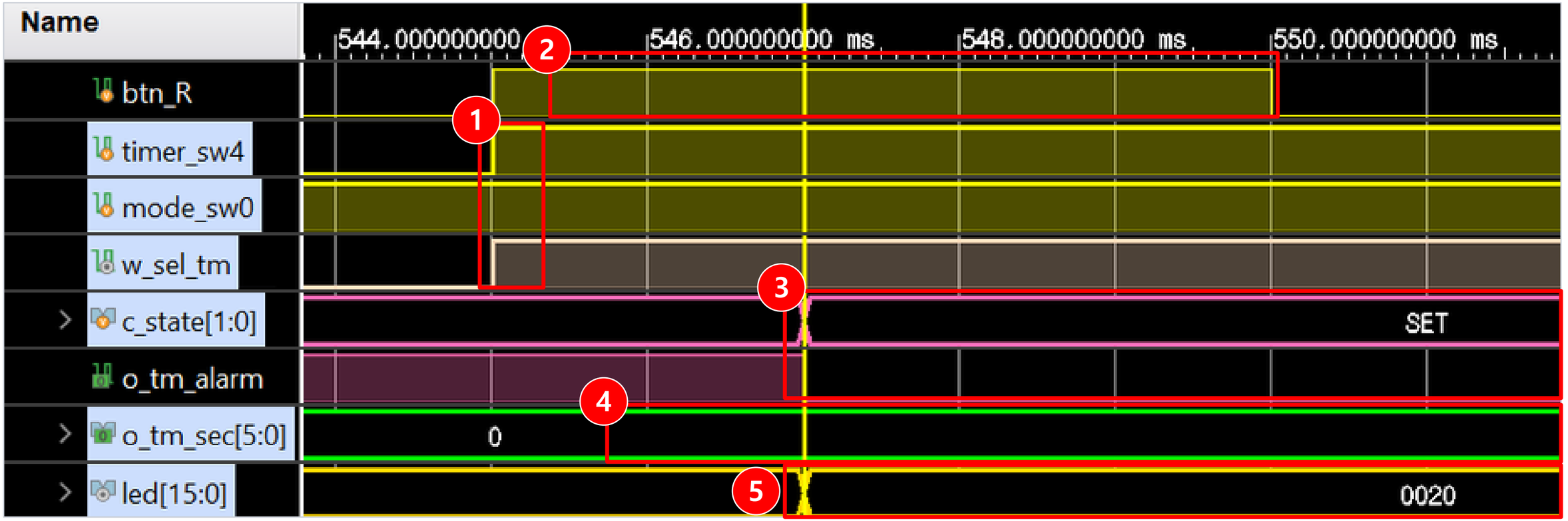
~150 ms sw4 → 0 (시계 화면) — **약 310 ms 동안 화면은 시계, 타이머는 계속 감소** → **~460 ms** 0 → ALARM

Figure 10. 타이머 백그라운드 동작 (왼쪽 : 화면 전환, 오른쪽 : 알람) — Vivado xsim 파형

부록 타이머 검증 상세

시나리오 5-2 : 화면 우선순위 · 알람 끄기

결과 **sw4 · sw0 모두 1 → 타이머 화면, 알람 중 R 은 알람만 끄고 값 유지**



① sw4 1 (sw0 도 1) → 타이머 화면 ② R 누름 ③ **ALARM → SET, alarm 0**

④ **값 그대로 (1분 안 더함)** ⑤ LED 점멸 끝 (0020)

Figure 11. 타이머 화면 우선순위 · 알람 끄기 — Vivado xsim 파형

부록 타이머 검증 상세

타이머 검증 신뢰도 — 변이 테스트

결과 **bug 2 종 주입 → TB 가 FAIL 7 개 · 1 개로 검출, 원 설계 MISMATCH 0**

일부러 넣은 bug	TB	결과
timer_datapath: 59:59 상한 조건을 끔	TM1	**FAIL 7개**
timer_control_unit: RUN 에서 U 를 받아 +10초	TM3	**FAIL 1개**
원래 설계	TM1 · TM3	**MISMATCH 0**

```
# bug 넣은 설계로 돌린 콘솔
[310015010 ns] FAIL P5 TB1 59:50 + U exp 59:59.00 got 60:0.0
TM1 checks = 21, TOTAL MISMATCH = 7
[50003010 ns] FAIL P1 TC2 U/R/D during RUN exp (5989,5999) got 6996
TM3 checks = 11, TOTAL MISMATCH = 1
```

→ MISMATCH 0 은 "검사가 엉성해서 0" 이 아니라 설계가 사양대로라는 뜻